TCP/IP Model and Wireshark Practical — Answers

Note: Wireshark results depend on your computer, browser, and network. The packet examples below are sample observations. For Question 1 and Question 5, use your own capture and replace the sample details with what you actually see.

1. Capture Flipkart traffic and identify packets from four layers

Procedure

1. Open Wireshark and select the network interface that is carrying your internet traffic (usually Wi-Fi or Ethernet).

2. Start capturing, open a browser, and refresh the Flipkart homepage. Wait a few seconds, then stop the capture.

3. In the display filter box, try `dns`, `tcp`, `tls`, or `http` if visible. Modern HTTPS websites usually encrypt application content, so you may see TLS rather than readable webpage data.

4. Click a packet and expand the packet details pane. Examine the protocol headers and record one example for each layer.

TCP/IP layer	Example packet / header to inspect	How to identify it	What it does
Application	DNS query/response, TLS, or HTTP if present	Look for DNS, TLS, HTTP, or another application protocol in the packet details.	Supports a browser service, such as resolving a website name or setting up encrypted HTTPS communication.
Transport	TCP segment (or UDP datagram)	Expand **Transmission Control Protocol** or **User Datagram Protocol**; inspect source and destination ports.	TCP manages reliable, ordered delivery; ports help identify the sending and receiving application services.
Internet	IPv4 or IPv6 packet	Expand **Internet Protocol Version 4** or **Internet Protocol Version 6**; note source and destination IP addresses.	IP addresses and routing help carry data between your device and remote networks.
Network Access	Ethernet II or IEEE 802.11 frame	Expand **Ethernet II** on a wired capture, or **802.11** on a suitable wireless capture; inspect MAC addresses when available.	Moves the frame over the local link using link-layer addressing and frame checks.

Important: One captured frame can show several layers at once. For example, an Ethernet frame may contain an IP packet, which contains a TCP segment, which carries TLS data. A single packet is therefore often a good way to inspect multiple headers.

2. OSI and TCP/IP comparison with app examples

The OSI model has seven layers. The commonly taught TCP/IP model has four layers; its Application layer combines OSI Application, Presentation, and Session, while its Network Access layer combines OSI Data Link and Physical.

OSI layer	TCP/IP layer	Purpose	Real-world app example
7. Application	Application	Network services used by apps	Opening a Zomato restaurant page or sending a WhatsApp message
6. Presentation	Application	Data format, encryption, compression	HTTPS/TLS encrypts browser traffic; images are encoded in formats such as JPEG
5. Session	Application	Starts, manages, and ends communication sessions	Maintaining an authenticated shopping session while browsing Flipkart
4. Transport	Transport	Ports; reliability and delivery between applications	TCP carries web traffic; QUIC/UDP may carry traffic for modern apps
3. Network	Internet	Logical addressing and routing	IP packets travel from your phone toward Instagram servers

OSI layer	TCP/IP layer	Purpose	Real-world app example
2. Data Link	Network Access	Frames and local-link MAC addressing	Wi-Fi frame carries data from your phone to your router
1. Physical	Network Access	Signals over radio, copper, or fiber	Wi-Fi radio waves or Ethernet electrical signals carry bits

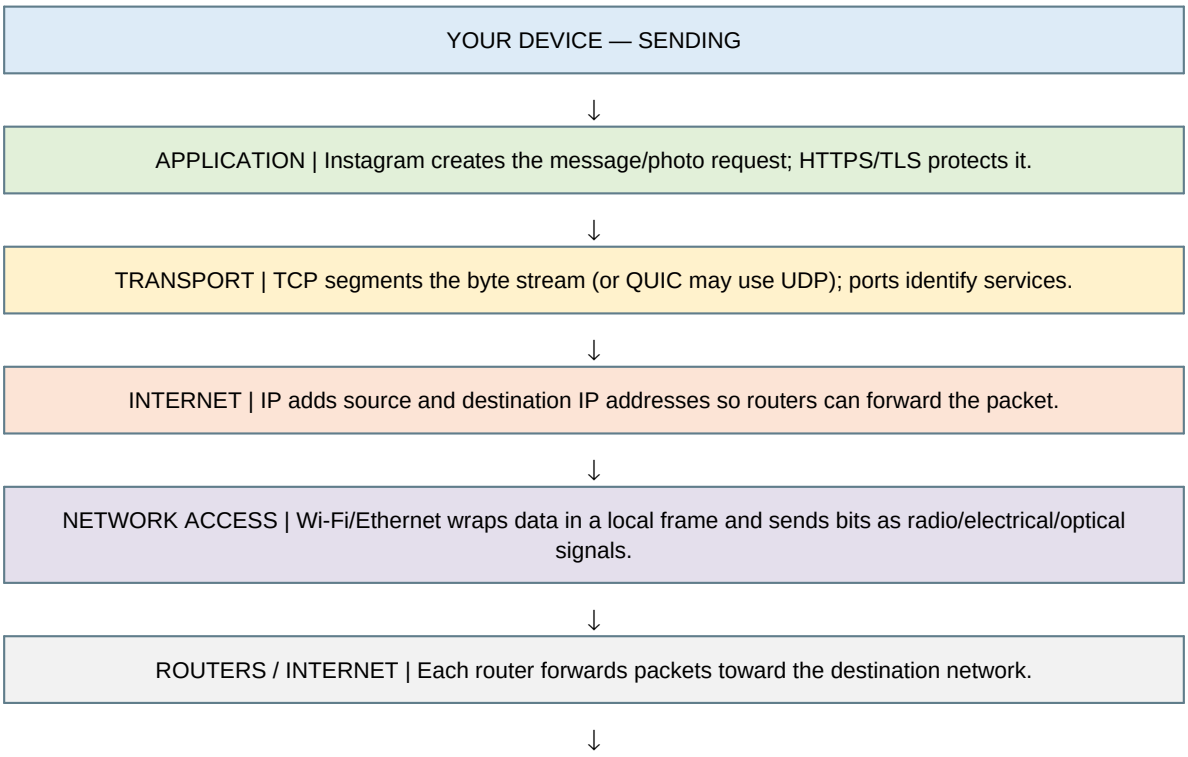
3. TCP segment header fields and their functions

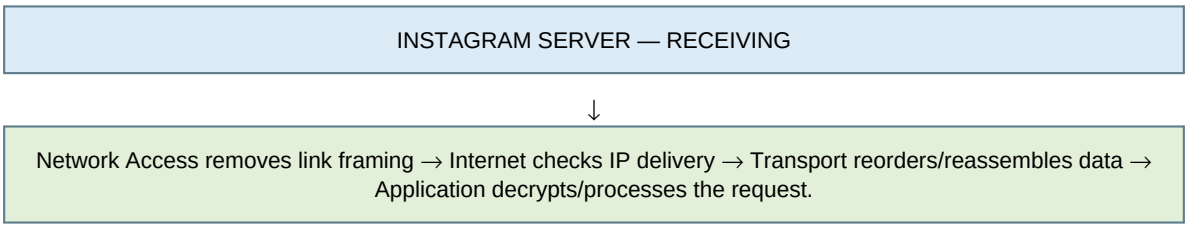
When the trainer provides the screenshot, label the fields shown in that exact screenshot. A typical TCP header contains the following fields:

TCP field	Function (1–2 lines)
Source Port	Identifies the sending application/service port on the source device.
Destination Port	Identifies the receiving application/service port on the destination device.
Sequence Number	Numbers bytes in the TCP data stream so the receiver can put data in the correct order and detect missing data.
Acknowledgment Number	When the ACK flag is set, indicates the next byte expected from the other side.
Header Length / Data Offset	Shows where the TCP payload begins by specifying the TCP header length.
Flags / Control Bits	Bits such as SYN, ACK, FIN, RST, PSH, and URG control connection setup, acknowledgments, closing, resets, and data handling.
Window Size	Advertises how much data the sender is currently prepared to receive; supports flow control.
Checksum	Helps detect corruption in the TCP header and data during transmission.
Urgent Pointer	Used with the URG flag to indicate urgent data; uncommon in typical modern web traffic.
Options and Padding	Optional settings such as Maximum Segment Size (MSS) and window scaling; padding aligns the header.
Data / Payload	The application data carried by the TCP segment; with HTTPS this content is normally encrypted.

4. Diagram: a message travels from your device to Instagram

Example: you send a message or upload a photo in Instagram. The simplified flow below shows the four TCP/IP layers as data moves down your device's stack, across the network, and up the server's stack.





Note: TCP/IP is a layered model, but the exact transport protocol depends on the app and connection. Instagram traffic is usually encrypted; do not expect to read message contents in an ordinary Wireshark capture.

5. Example packet analysis

Sample observation (replace this with your actual packet details):

I captured a DNS query while refreshing the Flipkart homepage. I identified it as an **Application-layer** packet because Wireshark labelled the protocol as DNS and showed a queried domain name. The action it represents is my browser asking a DNS resolver to find the IP address associated with a website name, allowing the browser to connect to the site.

How to write your own result: “Packet No. ___ uses the _____ protocol. I identified it by expanding the _____ header in Wireshark. It belongs to the _____ TCP/IP layer. It represents _____ in the app.”

If your capture does not show DNS, you can choose a TLS packet and explain that it represents encrypted HTTPS communication between the browser and the website. Do not claim that a packet proves a specific button click unless the packet details actually support that conclusion.